

Quiz — 2.2.2 Computational Methods

Name: _____ Date: _ **Mark:** ____ / 25

OCR H446 · Component 02 · 2.2.2

Section A — Multiple choice (8 marks)

A1. Which best describes a problem that is **solvable computationally**? A. Any problem a human can describe in words B. A problem with a clear, finite sequence of rule-based steps that can be decomposed C. Only problems that involve numbers D. A problem that always has exactly one solution Your answer: ☐

A2. Decomposition means: A. Hiding unnecessary detail B. Breaking a complex problem into smaller, manageable sub-problems C. Trying options and backing up at dead ends D. Predicting performance with a model Your answer: ☐

A3. Divide and conquer is most directly associated with which technique? A. Linear search B. Recursion (splitting into smaller instances of the same problem) C. Pipelining D. Data mining Your answer: ☐

A4. A satnav giving a fast, **good-enough but not guaranteed optimal** route is using a: A. Heuristic B. Brute-force search C. Backtracking algorithm D. Visualisation Your answer: ☐

A5. Solving a maze by going back to the last junction after hitting a dead end is: A. Abstraction B. Pipelining C. Backtracking D. Performance modelling Your answer: ☐

A6. Searching very large data sets for hidden patterns and correlations is: A. Visualisation B. Data mining C. Decomposition D. Heuristics Your answer: ☐

A7. Splitting a process into stages so different stages work on **different items at the same time** is: A. Backtracking B. Pipelining C. Abstraction D. Divide and conquer Your answer: ☐

A8. The Tube map removing exact distances and showing only line connections is an example of: A. Performance modelling B. Representational abstraction C. Data mining D. Backtracking Your answer: ☐

Section B — Short answer (12 marks)

B1. State **two** characteristics that make a problem solvable computationally. [2]

B2. Match each computational method (1–5) to the most appropriate use (A–E). Write the letter next to each number. [3]

#	Method		Use
1	Backtracking	A	Predicting how a server copes with load before it is built
2	Data mining	B	Solving Sudoku by undoing a guess at a dead end
3	Heuristics	C	Loyalty-card basket analysis to find buying patterns
4	Performance modelling	D	Representing infection rates as a colour-coded heat map
5	Visualisation	E	A* route-finding giving a quick near-optimal answer

1 → _ 2 → 3 → _ 4 → 5 → _____

B3. Explain the difference between **abstraction by representation** and **abstraction by generalisation**, giving one example of each. [2]

B4. A company wants to model the spread of a delivery route over a road network. Explain how **divide and conquer** could be applied, and state why it pairs naturally with **recursion**. [2]

B5. A heuristic is used to solve the Travelling Salesman Problem. Explain **why** a heuristic is used here rather than an exact method. [2]

B6. State **one** benefit of **visualisation** when analysing large data sets. [1]

Section C — Exam-style question (5 marks)

A games company is writing a program to place 8 queens on a chessboard so that no two attack each other (the N-Queens problem).

(a) Name the computational method best suited to this problem. [1]

(b) Describe **how** the chosen method would attempt to find a valid arrangement, including what happens at a **dead end**. [3]

(c) State **one** reason this method may still be slow for large boards. [1]

Answer key

Section A (1 mark each = 8)

Q	Ans	Why
A1	B	A computationally solvable problem can be decomposed and expressed as a clear, finite, rule-based sequence of steps.
A2	B	Decomposition = breaking a complex problem into smaller, manageable sub-problems.
A3	B	Divide and conquer splits into smaller instances of the same problem and combines results — naturally recursive.
A4	A	A heuristic gives a quick good-enough (not guaranteed optimal) answer, e.g. satnav A*.
A5	C	Returning to the last decision point after a dead end is backtracking .
A6	B	Searching large data sets for hidden patterns/correlations is data mining .
A7	B	Pipelining overlaps stages so different items are processed simultaneously (e.g. CPU fetch–decode–execute).
A8	B	Dropping irrelevant detail (exact distances) while keeping structure is representational abstraction .

Section B (12)

B1. [2] Any two (1 each): can be **decomposed** into sub-problems; has clear **inputs, processes and outputs**; is repetitive / rule-based (algorithmic); allows useful **abstractions**; data can be **structured/modelled**; has a clear **finite** sequence of steps.

B2. [3] (½ mark each, 6 correct = 3 marks; round to whole marks: 5–6 correct = 3, 3–4 = 2, 1–2 = 1) - 1 → **B** (backtracking → Sudoku dead end) - 2 → **C** (data mining → basket analysis) - 3 → **E** (heuristics → A near-optimal) - 4 → **A** (performance modelling → server load) - 5 → **D*** (visualisation → heat map)

B3. [2] - Representational abstraction (1): remove/hide detail not relevant to the problem, keeping only what is needed — e.g. the London Tube map keeps line connections but drops true distances/geography.
- **Abstraction by generalisation (1):** group things by **shared characteristics** so they can be treated the same — e.g. classifying many shapes as "polygon", or grouping car types as "vehicle".

B4. [2] - 1 mark: split the network/route into smaller sub-sections (smaller instances of the same routing problem), solve each part, then **combine** the partial results into the full route. - 1 mark: it pairs with **recursion** because each smaller instance is the **same type of problem**, so the same subroutine can call itself on the sub-parts until a base case (a trivially small section) is reached.

B5. [2] - 1 mark: an exact method must check an enormous number of route combinations — it is **intractable** / grows factorially, so it is too slow for many cities. - 1 mark: a **heuristic** uses a rule of thumb to find a **good-enough** route quickly, accepting it may **not be the optimal** one.

B6. [1] Any one: patterns/trends/correlations/outliers become easier to spot than in raw figures; aids quicker decision-making; communicates findings clearly.

Section C (5)

(a) **[1] Backtracking** (accept "recursive backtracking").

(b) [3] Up to 3 from: - Place queens one at a time, column by column (or row by row), choosing a safe square for each (1). - After each placement, check the partial arrangement is still valid (no two queens attack) (1). - On reaching a **dead end** — no safe square in the next column — **backtrack**: undo the last placement and try the next available square for the previous queen (1). - Repeat until all 8 are placed (a solution) or all options are exhausted (1).

(c) [1] Any one: the search space grows very large / many combinations must be explored; lots of backtracking is needed as the board size increases (combinatorial explosion / intractable for large N).

Total: $8 + 12 + 5 = 25$.